CAD for VLSI Design

指導教授: 陳聿廣

PA4

Analog Floorplan

(結報)

系級 電機四 B

學號 110501544

姓名 翁瑄妤

目錄

1. Compilation and Execution 3

2. Completion 3

2.1 The Result of Execution..... 3

2.2 The Result Diagram 4

3. Algorithms and Data Structures 5

3.1 Overall Framework 5

3.2 Data Structure Design 6

3.3 Algorithm Design 7

4. Perturbation Strategy and Operations 8

4.1 Perturbation Strategy Overview 9

4.2 Variant Switch 9

4.3 Node Relocation 9

5. Cost Function and State Transition 10

5.1 Cost Function Design 10

5.2 State Transition Decision 10

5.3 Temperature Control Strategy 10

6. Hardness 11

7. Suggestions 11

1. Compilation and Execution

主要在工作站上執行程式，其系統為 Linux 3.10.0、Host 為 140.115.71.87。

執行指令分為以下步驟:

1. 打開所屬資料夾 (所需資料皆在 PA4 的資料夾裡)

   cd PA4/

2. 執行 Makfile 檔案(有另外加 run1、run2、check1、check2 做測試用)：

```makefile
# PA4 cpp compiler
.PHONY: all run run1 run2 check1 check2 draw1 draw2 clean
all: main.cpp src/Contour.cpp src/Floorplanner.cpp src/SimulatedAnealing.cpp
	g++ -std=c++11 main.cpp src/Contour.cpp src/Floorplanner.cpp src/SimulatedAnealing.cpp -o exe
run:
	./exe $(input) $(output)
run1:
	./exe case1.block case1.out
run2:
	./exe case2.block case2.out
check1:
	./PA4_checker case1.block case1.out
check2:
	./PA4_checker case2.block case2.out
draw1:
	python visualizer.py case1.out
draw2:
	python visualizer.py case2.out
clean:
	rm exe
	rm -rf *.txt
```

   - 將作業 cpp 檔輸出成執行檔.o 檔

     make all

   - 執行輸出檔，生成輸出檔案(以 case1.block 為例)

     make run input=case1.block output=case1.txt

   - 清除執行檔

     make clean

3. 執行 CHECKER 檔確認輸出檔案內容是否正確 (以 c17 為例)

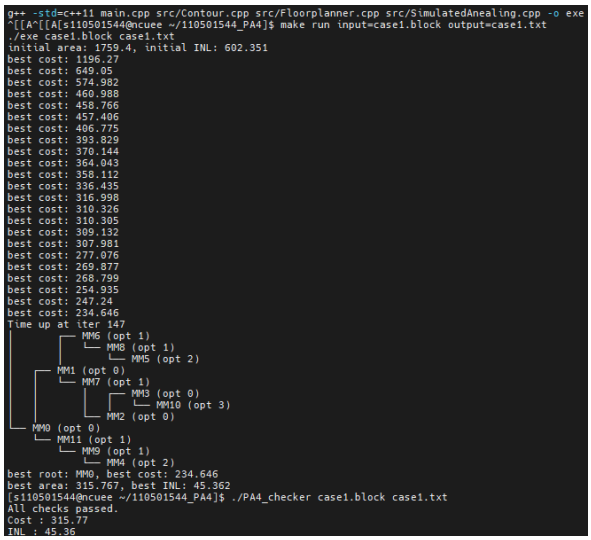
   chmod +x PA4_checker

   ./PA4_checker case1.block case1.txt

2. Completion

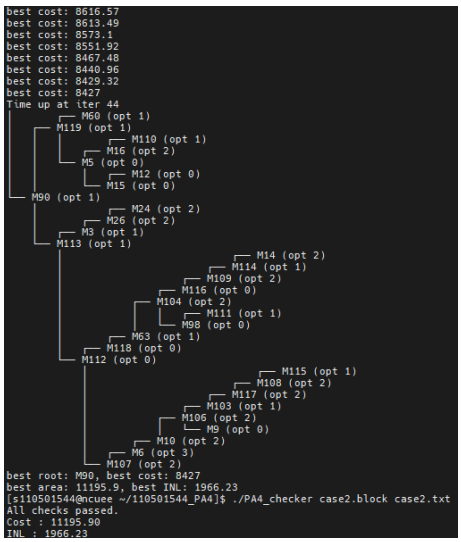
2.1 The Result of Execution

case1.block



```
g++ -std=c++11 main.cpp src/Contour.cpp src/Floorplanner.cpp src/SimulatedAnealing.cpp -o exe
^[[A^[[A[s110501544@ncuee ~/110501544_PA4]$ make run input=case1.block output=case1.txt
./exe case1.block case1.txt
initial area: 1759.4, initial INL: 602.351
best cost: 1196.27
best cost: 649.05
best cost: 574.982
best cost: 460.988
best cost: 458.766
best cost: 457.406
best cost: 406.775
best cost: 393.829
best cost: 370.144
best cost: 364.043
best cost: 358.112
best cost: 336.435
best cost: 316.998
best cost: 310.326
best cost: 310.305
best cost: 309.132
best cost: 307.981
best cost: 277.076
best cost: 269.877
best cost: 268.799
best cost: 254.935
best cost: 247.24
best cost: 234.646
Time up at iter 147
│       ┌── MM6 (opt 1)
│       │   └── MM8 (opt 1)
│       │       └── MM5 (opt 2)
│   ┌── MM1 (opt 0)
│   │   └── MM7 (opt 1)
│   │       │   ┌── MM3 (opt 0)
│   │       │   │   └── MM10 (opt 3)
│   │       └── MM2 (opt 0)
└── MM0 (opt 0)
    └── MM11 (opt 1)
        └── MM9 (opt 1)
            └── MM4 (opt 2)
best root: MM0, best cost: 234.646
best area: 315.767, best INL: 45.362
[s110501544@ncuee ~/110501544_PA4]$ ./PA4_checker case1.block case1.txt
All checks passed.
Cost : 315.77
INL : 45.36
```

case2.block



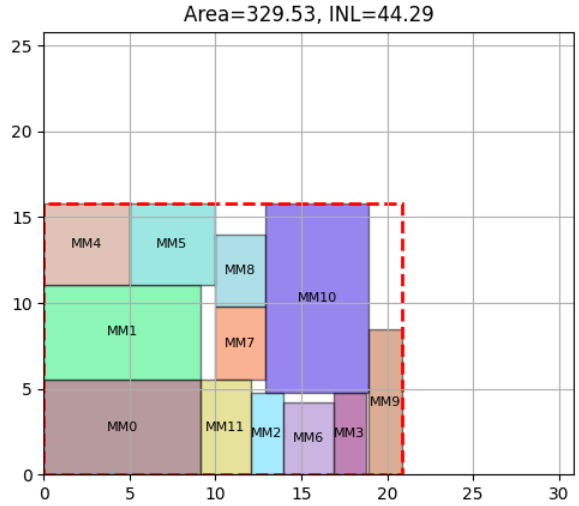
```
best cost: 8616.57
best cost: 8613.49
best cost: 8573.1
best cost: 8551.92
best cost: 8467.48
best cost: 8440.96
best cost: 8429.32
best cost: 8427
Time up at iter 44
│       ┌── M60 (opt 1)
│   ┌── M119 (opt 1)
│   │   │       ┌── M110 (opt 1)
│   │   │   ┌── M16 (opt 2)
│   │   └── M5 (opt 0)
│   │       │   ┌── M12 (opt 0)
│   │       └── M15 (opt 0)
└── M90 (opt 1)
    │       ┌── M24 (opt 2)
    │   ┌── M26 (opt 2)
    │   │   ┌── M3 (opt 1)
    └── M113 (opt 1)
        │                       ┌── M14 (opt 2)
        │                   ┌── M114 (opt 1)
        │               ┌── M109 (opt 2)
        │           ┌── M116 (opt 0)
        │       ┌── M104 (opt 2)
        │       │   │   ┌── M111 (opt 1)
        │       │   └── M98 (opt 0)
        │   ┌── M63 (opt 1)
        │   │   ┌── M118 (opt 0)
        └── M112 (opt 0)
            │                       ┌── M115 (opt 1)
            │                   ┌── M108 (opt 2)
            │               ┌── M117 (opt 2)
            │           ┌── M103 (opt 1)
            │       ┌── M106 (opt 2)
            │       │   └── M9 (opt 0)
            │   ┌── M10 (opt 2)
            │   │   ┌── M6 (opt 3)
            └── M107 (opt 2)
best root: M90, best cost: 8427
best area: 11195.9, best INL: 1966.23
[s110501544@ncuee ~/110501544_PA4]$ ./PA4_checker case2.block case2.txt
All checks passed.
Cost : 11195.90
INL : 1966.23
```

由上面兩張照片可得知，執行出來的兩份輸出檔案皆為正確，且指令皆以比照投影片要求執行，因此這份檔案已完成作業中的所有要求。

2.2 Result diagram

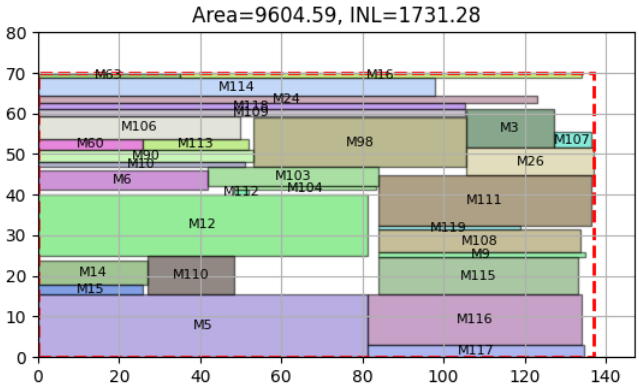
1. case1.block

```
Time up at iter 3518
│           ┌─ MM4 (opt 2)
│           │  └─ MM5 (opt 2)
│           │     │  ┌─ MM8 (opt 1)
│           │     └─ MM7 (opt 1)
│           │        └─ MM10 (opt 3)
│           │           └─ MM9 (opt 0)
│     ┌─ MM1 (opt 0)
└─ MM0 (opt 0)
      └─ MM11 (opt 1)
         └─ MM2 (opt 0)
            └─ MM6 (opt 1)
               └─ MM3 (opt 0)
best root: MM0, best cost: 243.958
best area: 329.531, best INL: 44.2891
```



2. case2.block

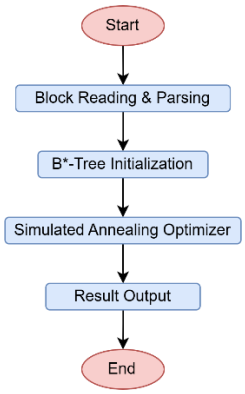
```
Time up at iter 989
│                          ┌─ M63 (opt 2)
│                          │  └─ M16 (opt 2)
│                       ┌─ M114 (opt 1)
│                    ┌─ M118 (opt 2)
│                    │  └─ M107 (opt 0)
│                 ┌─ M106 (opt 1)
│                 │  └─ M3 (opt 0)
│              ┌─ M10 (opt 2)
│           ┌─ M12 (opt 0)
│        ┌─ M14 (opt 0)
│        │  └─ M112 (opt 0)
│        │        ┌─ M24 (opt 2)
│        │     ┌─ M111 (opt 0)
│        │     └─ M115 (opt 0)
│     ┌─ M15 (opt 1)
│     │        ┌─ M90 (opt 1)
│     │        │  └─ M98 (opt 0)
│     │     ┌─ M6 (opt 1)
│     │     │  └─ M9 (opt 2)
│     └─ M110 (opt 0)
│     │        ┌─ M109 (opt 2)
│     │     ┌─ M60 (opt 1)
│     │     └─ M104 (opt 2)
│     │        └─ M108 (opt 1)
└─ M5 (opt 0)
      │     ┌─ M113 (opt 1)
      │     │  └─ M26 (opt 0)
      │  ┌─ M103 (opt 1)
      │  │  └─ M119 (opt 2)
      └─ M116 (opt 0)
         └─ M117 (opt 1)
best root: M5, best cost: 7242.59
best area: 9604.59, best INL: 1731.28
```



在執行程式上，最終會繪出最好的 binary tree 架構解，並利用 python 視覺化每個 Block 實際放置的位置。

3. Algorithms and Data Structures

3.1 Overall Framework



整體架構利用 class 模組化設計，主要包含以下幾個模組：

1. Contour 模組
   - 管理「水平」與「垂直」兩條輪廓線，用來追蹤已放置硬塊的 occupied profile
   - 確保新放置的硬塊不會與現有區塊重疊
   - 作為 floorplanning 時，找到最左下（或最貼合父節點）的位置基準

```
// Contour class for maintaining skyline/contour information during packing
class Contour {
private:
    // Structure to represent a segment of the contour
    struct Segment {
        double start_x, end_x, height;      // Start x, end x, and height of segment
        Segment(double s, double e, double h) : start_x(s), end_x(e), height(h) {}
    };

    std::vector<Segment> segments;          // List of contour segments

public:
    // Constructor initializes with a base segment covering infinite width at height 0
    Contour();

    // Get maximum height in the range [x_start, x_start + width]
    double getMaxHeight(double x_start, double width);

    // Update contour by placing a rectangle at (x_start, width) with new_height
    void update(double x_start, double width, double new_height);

    // Get maximum x coordinate where height > 0
    double maxX() const;

    // Get maximum height across all segments
    double maxY() const;
};
```

2. Floorplanner 模組
   - 保存整棵 B*-Tree 結構與所有 Block 的尺寸與座標
   - 以層序或遞迴方式，根據 B*-Tree 拓撲決定每個 Block 放置順序
   - 透過 Contour 提供的位置資訊，為每個 Block 計算最合法的放置座標
   - 評估品質（面積、INL 等）並提供擾動方法

```cpp
// Main floorplanning class that manages B*-Tree and block placement
class Floorplanner {
public:
    std::vector<Block> blocks;      // List of blocks to be placed
    BTreeNode* root;                // Root of the B*-Tree
    Contour hcontour;               // Horizontal contour for placement
    Contour vcontour;               // Vertical contour for placement

    // Default constructor
    Floorplanner(const std::vector<Block>& bks);
    // Copy constructor for deep copying
    Floorplanner(const Floorplanner& o);
    // Assignment operator
    Floorplanner& operator=(const Floorplanner& o);
    // Destructor to clean up memory
    ~Floorplanner();
    // Build initial B*-Tree structure from blocks
    void buildInitialBTree();
    // Place blocks according to B*-Tree structure using contour packing
    void place();
    // Calculate total area of the floorplan (bounding box area)
    double area() const;
    // Calculate Integral Non-Linearity (INL) metric
    double INL() const;
    // Calculate centroid of the floorplan
    std::pair<double, double> centroid() const;
    // Calculate penalty for overlapping blocks
    double overlapPenalty() const;
    // Perform perturbation for simulated annealing (swap, variant change, move)
    void perturb();
    // Check if target node is descendant of root node
    bool isDescendant(BTreeNode* root, BTreeNode* target);
    // Print B*-Tree structure for debugging
    void printTree(BTreeNode* node, const std::string& prefix = "", bool isRight = true);

private:
    // Helper function to clone entire tree structure
    BTreeNode* cloneTree(BTreeNode* node);
    // Helper function to recursively delete tree
    void deleteTree(BTreeNode* node);
    // Helper function to swap block indices in B*-Tree nodes
    void swapBlockIndex(BTreeNode* node, int a, int b);
    // Collect all nodes in the tree into a vector
    void collectNodes(BTreeNode* node, std::vector<BTreeNode*>& nodes);
    // Remove a specific node from the tree
    bool removeNode(BTreeNode*& node, BTreeNode* target);
    // Insert a subtree at the rightmost position of given root
    void insertSubtree(BTreeNode* root, BTreeNode* subtree);
};
```

3. SimulatedAnnealing 模組
   - 負責全域搜尋策略，包裹 Floorplanner 的擾動與評估
   - 控制「溫度」與「降溫／重新加熱」機制，平衡探索與利用
   - 根據成本函數（面積＋長寬比、INL 等加權）決定接受新解的機率
   - 維護目前解與最佳解，並在時間或迭代上限後輸出最終配置

```cpp
// Simulated Annealing optimization class for floorplanning
class SimulatedAnnealing {
public:
    // Constructor with optimization parameters
    SimulatedAnnealing(Floorplanner initial, int max_iter = 1000000,
                       double init_temp = 100.0, double alpha = 0.7);

    // Run the simulated annealing optimization process
    void run();

    // Get the best solution found during optimization
    Floorplanner getBest();

private:
    Floorplanner best;          // Best solution found so far
    Floorplanner current;       // Current solution in optimization
    Floorplanner Initial;       // Initial solution for reference
    double best_cost;           // Cost of the best solution
    int ITER;                   // Maximum number of iterations
    double T;                   // Initial temperature
    double A;                   // Alpha parameter for cost function weighting
    double area_ref, inl_ref;   // Reference values for normalization
    double total_time;
    std::chrono::steady_clock::time_point deadline;  // Time limit for optimization
    std::chrono::steady_clock::time_point start_time;  // Time limit for optimization

    // Calculate cost function combining area, aspect ratio, and INL
    double cost(const Floorplanner& fp) const;

    // Determine whether to accept a worse solution based on temperature
    bool accept(double new_cost) const;
};
```

3.2 Data Structure Design

1. struct BlockOption

用於儲存每個電晶體元件的不同尺寸選項，包括寬度、高度、列倍數和行倍數。

```
// Structure to define different size options for a block
struct BlockOption {
    double width, height;       // Dimensions of the block option
    int col_mult, row_mult;     // Column and row multipliers for layout
};
```

2. struct Block

代表電路中的一個電晶體元件，包含名稱、多種尺寸選項、目前的選項以及放置座標。

```
// Structure representing a block in the floorplan
struct Block {
    std::string name;                           // Name/identifier of the block
    std::vector<BlockOption> options;           // Available size options for this block
    int selected_option = 0;                    // Currently selected option index
    double x = 0, y = 0;                       // Position coordinates in the floorplan

    // Get width of currently selected option
    double width() const { return options[selected_option].width; }

    // Get height of currently selected option
    double height() const { return options[selected_option].height; }
};
```

3. struct BTreeNode

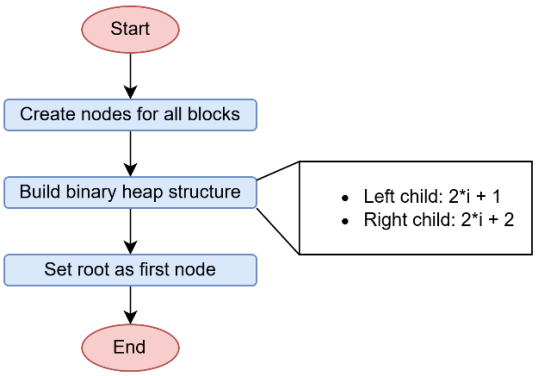
B*-Tree 的節點結構，left 指向右方的子節點，right 指向上方的子節點。

```
// Node structure for B*-Tree representation
struct BTreeNode {
    int block_idx;              // Index of the block this node represents
    BTreeNode* left;            // Right child (represents blocks to the right)
    BTreeNode* right;           // Upper child (represents blocks above)

    // Constructor to initialize node with block index
    BTreeNode(int idx) : block_idx(idx), left(nullptr), right(nullptr) {}
};
```

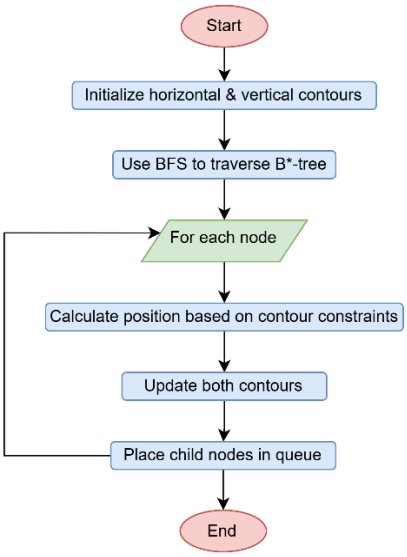
3.3 Algorithm Design

1. B*-Tree Initialization Algorithm (buildInitialBTree())



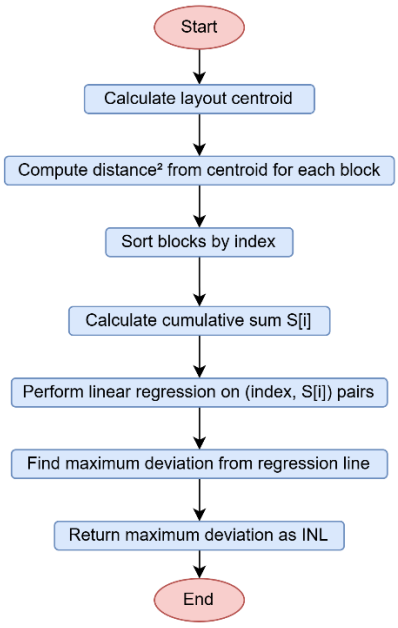
將所有區塊組織成一個類似完全二元樹的結構，每個節點代表一個區塊，左子樹表示右方相鄰的區塊，右子樹表示上方相鄰的區塊。

2. Contour-based Packing Algorithm (place())



使用兩條輪廓線（水平和垂直）來追蹤已放置區塊的邊界，確保新放置的區塊不會重疊。

3. INL Calculation Algorithm (INL())



對所有 block 中心距質心的累積非線性誤差進行積分，對後續計算 cost 值時有幫助。

4. Perturbation Strategy and Operations

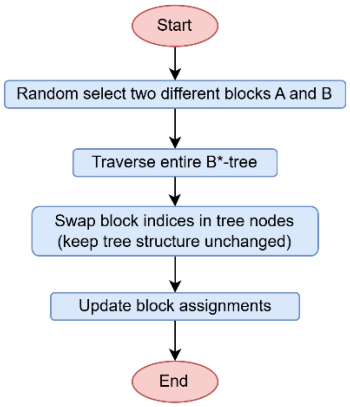
模擬退火過程中，主要以 B*-Tree 為基礎結構，使用三種主要的擾動操作來生成鄰域解，並隨機等機率選擇其中一種執行，確保搜索空間的多樣性與廣度。

擾動策略	機率
Block Index Swap	33.3%

Variant Switch	33.3%
Node Relocation	33.3%

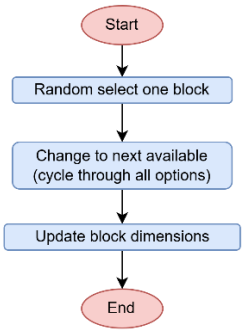
4.1 Perturbation Strategy Overview

目標：保持 B*-Tree 結構不變，只交換兩個節點所代表的物理區塊，以改變它們在 Floorplan 中的相對位置關係。



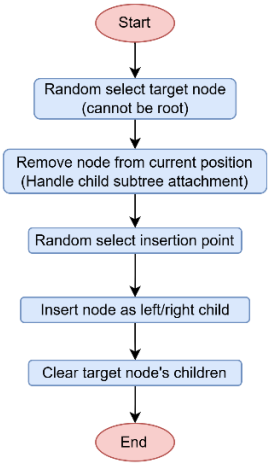
4.2 Variant Switch

目標：在同一個 block 上切換寬高配比選項，讓 Block 能在多種尺寸下重組布局。



4.3 Node Relocation

目標：真正改變 B*-Tree 的結構，將某個節點從樹中移除並重新插入到另一處，以獲得更大范圍的拓撲變化。



5. Cost Function and State Transition

5.1 Cost Function Design

主要將「面積最佳化」與「INL 最小化」兩大目標合併成單一數值，用於排序與接受判斷。

$$\text{Total Cost} = \alpha \times \text{Area_Cost} + (1 - \alpha) \times \text{INL_Cost}$$

- α：面積成本權重（$0 \le \alpha \le 1$），實驗調整。
- Area_Cost：結合 bounding-box 面積與長寬比罰則：

$$\text{Area_Cost} = \text{Chip_Area} \times [1 + F(AR)]$$

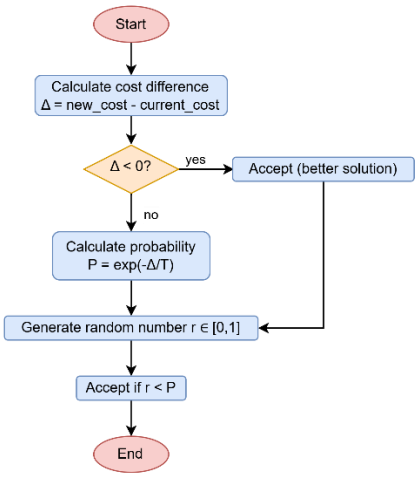
  - Chip_Area = W×H，其中 $W = \max_i (x_i + w_i)$，$H = \max_i (y_i + h_i)$。
  - AR = max(W/H, H/W)
  - 罰則函數：

$$F(AR) = \begin{cases} 2(0.5 - AR), & 0 \le AR < 0.5 \\ 0, & 0.5 \le AR \le 2 \\ (AR - 2), & AR > 2 \end{cases}$$

- INL_Cost = INL：對所有 block 中心距質心的累積非線性誤差進行積分，見 Section 3.3。

5.2 State Transition Decision

accept() function :



5.3 Temperature Control Strategy

採用多階段分段冷却，並配合自適應重熱機制，根據已用時間比例進行降溫：

時間階段	冷卻係數	說明
前 40%（0–236 s）	0.98	快速降溫，粗略搜尋區域
中 30%（236–413 s）	0.995	慢速降溫，精細搜尋
後 30%（413–590 s）	0.999	極慢降溫，微調最佳解

6. Hardness

1. B*-Tree 結構

一開始以為調整 B*-Tree 拓撲就只是換一下 parent 或 child 指標，結果根本沒這麼簡單。移一個節點，下面整棵子樹的關係都要重新整理，如果沒處理好，就會整個斷樹 or 重複節點超亂。

後來是自己慢慢拆成幾個功能來搞定的：

- removeNode() 負責安全地拔掉一個節點，還要顧好它的小孩怎麼接上去。
- insertSubtree() 則是把拔下來的節點加回去時，能正確插在新位置。
- 最後寫了個 collectNodes()，方便收集所有節點來選擇要操作哪一個，避免每次都得重新走一遍樹。

這三個搭配起來才終於讓 Node Relocation 不會亂掉。

2. Contour 維護

每次放一個 block 都要更新上方的輪廓線，但 Contour 是多段線組成的，要考慮重疊、切割、合併，光想就頭痛。

我最後是先自己定義一個 Segment 結構，把每段輪廓的 start, end, height 都存起來。

然後每次放新 block：

- 先找出會影響到的 segment
- 分割它們
- 插入新的區塊輪廓
- 最後把左右相連、同高度的段合起來，避免輪廓太破碎

加上每次更新完都重新排序一次，確保 segment 是照 x/y 座標排列。這樣 contour 才能正確維持每個 block 上方的空間高度。

7. Suggestions

雖然作業有提供測資，但實際上大家很難評估自己的結果到底表現得好不好，尤其是在調整參數或演算法時，完全沒有一個參考的標準可以依據，常常會變成盲目嘗試。建議未來可以提供一些助教或學長姐的示範結果（例如面積、INL 或 cost 值），讓大家有個合理的參考範圍，這樣也比較知道努力的方向。